TOAD CITY BALTIMORE · OUTSIDE-AI BRIEF · CLAUDE OPUS

Characters: thug gangs and the 17 bosses

Tasks C and D · lead character builder · drop-in three.js rig modules



Recommended setting. Use **Claude Opus** (the newest Opus available to you) with **extended thinking ON at high effort** (max for the hardest bosses: Mister Formstone, the vehicle bosses and Sugar Rush). Work on **one gang or one boss per session**, and self-test it in `viewer.html` (day, night, every pose, the budget readout) before you move on. Opus is the lead character builder.

Prepared for Brad Parlette, 11 October 2026. Everything is built procedurally in code; real Baltimore signage is redrawn on canvas, never copied from logo files. The starter kit (`toadcity_kit.js`, `viewer.html` and two example modules) comes as `toadcity_starter_kit.zip`. Its core is printed in the appendix in case the zip is missing.

Contents

1 · Project overview 3

2 · Style guide and quality bar 5

3 · Code starter and interface 7

4 · Task C: Thug gangs (5 district gangs) 9

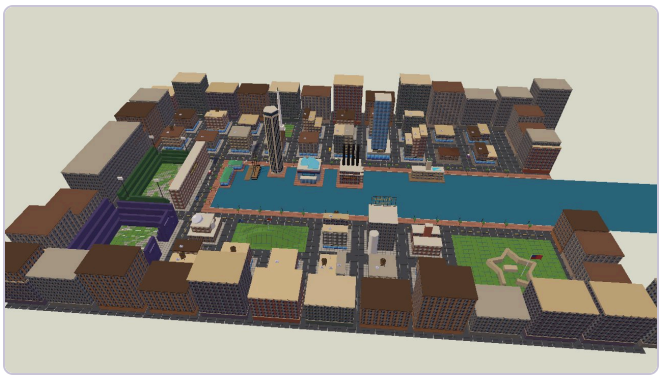
5 · Task D: The 17 bosses 13

6 · Delivery instructions 16

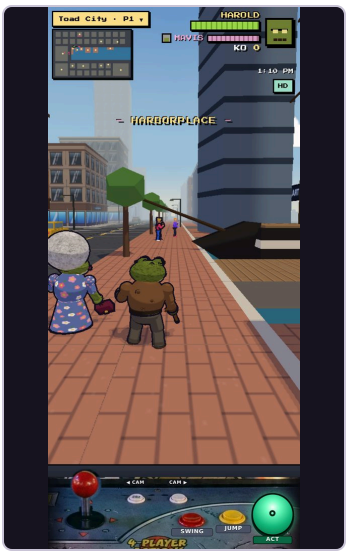
Appendix · Kit core (characters) 17

1 · Project overview

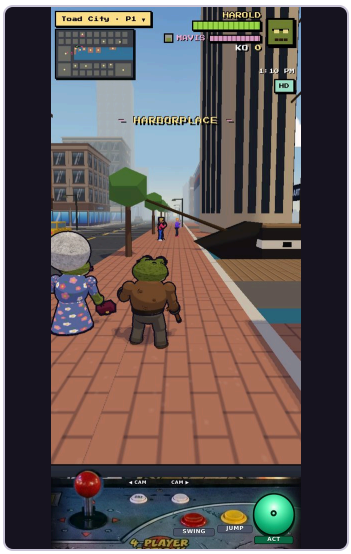
Toad City Baltimore is an open-world, cel-shaded street brawler set on Baltimore's Inner Harbor, built for phones first. You play **Harold**, an elderly toad with a cane. His wife **Mavis** (grey afro, floral dress, a handbag she swings) fights beside you under AI control. Together you clear crime out of each district around the harbour. Each landmark district has a set-piece fight and a boss, and the final boss is Mister Formstone. The real map is compressed about 3×: landmarks sit about one block apart, with a 6 m brick promenade along the water and land about two blocks deep. Day turns to night every 5 minutes (a 600 s loop), and at night the windows, street lamps and neon light up.



City overview, day: the harbour basin, the street grid and the landmark massings (in-house). Night lights about 45% of windows and warm lamps along the promenade.



iPhone portrait 390×844: the game view above the arcade control panel.



The arcade panel (ball-top stick, round buttons, big ACT button). Assets must read in the view above it.

Tech stack

- **three.js r160** with plain ES modules. The game source is a set of `src/*.js` files that a shell script concatenates into one module (`game.js`). There is no bundler and no framework.
- **Everything procedural:** geometry comes from three.js primitives in code, and every texture is drawn on a `<canvas>` at load. There are no image, model or font files to license. Audio is separate (not part of this work).
- **Look:** MeshLambert plus a 3-band toon ramp injected with `onBeforeCompile`, vertex colours, inverted-hull ink outlines and PCF soft shadows from one sun. A `?retro=1` 32-bit mode exists, but the kit is HD-only and that is what you deliver for.
- **How deliveries plug in:** each module is a file that receives the kit object `K` (the same functions the game uses). In-house, we strip the `export` keywords, append the file to `src/`, bind `K` to the game's globals and register the build function in the gang, boss, prop or vehicle tables. If your module runs in the kit viewer, it runs in the game.

Target devices and performance budgets

The main target is an **iPhone in portrait, 390×844 CSS px at DPR 2** (the game view is the tall area above the arcade panel), plus desktop Chrome and Safari. Aim for 60 fps, with a 30 fps floor on a mid-range iPhone. Current city frames measure **76–168 draw calls and 99k–346k triangles** at five spots (day, 390×844, including the shadow pass), so new assets must be lean.

Budget	Limit	Notes
Whole frame (iPhone)	≤ 250 draw calls, ≤ 400k tris	Includes the shadow pass. The in-house team owns this number; your assets must not blow it.
Reference: Harold / Mavis	60 / 54 parts · 14.0k / 15.7k tris	Measured in the kit viewer. Each part is a mesh plus an ink hull.
Reference: current thug	26–33 parts · 7.6k–8.5k tris	Two materials (one toon plus the shared outline).
New gang member (Task C)	≤ 40 parts · ≤ 12k tris · ≤ 3 mats	Far LOD (`opts.lod = 1`): ≤ 14 parts, ≤ 3k tris, no fine detail.
Boss (Task D)	≤ 70 parts · ≤ 22k tris · ≤ 4 mats	One boss on screen at a time.
Rowhouse kit (Task A)	house ≤ 2.5k · corner ≤ 3.5k · block ≤ 30k tris	Merged: a full block is ≤ 4 draw calls. One 1024² atlas for the whole kit.
Static props (Task B)	120–2,500 tris each	Merged into the chunk, so they add no draw calls. Per-prop limits are in the task.
Vehicles (Task B)	600–6,000 tris · 3–12 meshes	Their own Group, so they move. At most 2 materials each.
Textures	canvas only · ≤ 1024² atlas · ≤ 256² per character map	Drawn in code at load; mipmapped in HD.

Merged geometry and shared materials: the static city is written into merged builders, each 64 m chunk being one toon mesh, one outline hull, one glow mesh and one decal mesh, and the chunks are frustum-culled. Characters are groups of parts that share one toon material per rig. **LOD the game already does:** thugs beyond 24 m drop their ink hulls and stop casting shadows, and beyond 48 m they sleep (hidden). Hulls must therefore be BackSide ShaderMaterial children so this code can find them.

Coordinates and scale

Item	Value	Item	Value
Units / axes	metres · Y up · X east · Z south (north = −Z)	Facing at yaw 0	+Z (characters, vehicles, house fronts)
City block	24 × 24 m	Street	10 m wide (grid period 34 m)
Promenade	6 m brick strip on every water edge	Kerb / pavement top	`CURB = 0.16`
Ground floor / upper floor	`GF = 4.4` / `FH = 3.4`	Parapet	`PAR = 1.3`
Water surface	`WY = −0.85`	Render chunk / collision cell	64 m / 16 m
World bounds	x −136…213, z −106…104	Harbour basin	x −60…400, z −30…30
Harold / Mavis	about 1.6 m / about 2.0 m (afro)	Thug / bruiser	about 2.0 m / ×1.2

> **Signage rule.** Brad wants the city to look like the real Inner Harbor: storefronts, signs and props **may use real Baltimore businesses' names and sign styles where they really exist** (Phillips Seafood, Barnes & Noble and Hard Rock Cafe at the Power Plant, the water-taxi branding, Natty Boh-style corner-bar signs, Utz, Old Bay). The neon sign in section 2 shows how: the real script wording **redrawn on canvas**, never a copied logo file or photo. **Bosses and thugs stay original characters** (no real people).

Readability on a phone: silhouettes must read at 6–10 m on a 390×844 screen; exaggerate key shapes by 10–20% (heads, hands, cornices, stoops, lamp heads); give each gang, district or vehicle one accent colour; check every asset in both the day and the night preset before delivery.

2 · Style guide and quality bar

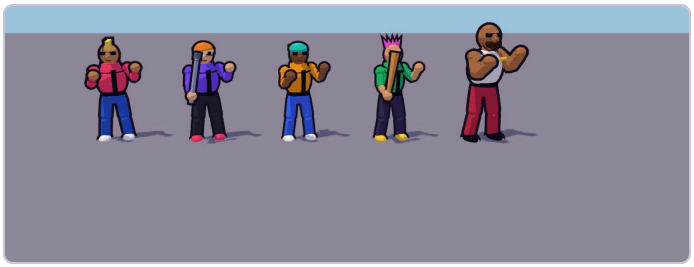
Everything you deliver must look like it belongs next to Harold and Mavis below. These are the **asset starters**: renders of the real game models through the kit viewer you were given. They are the quality bar.



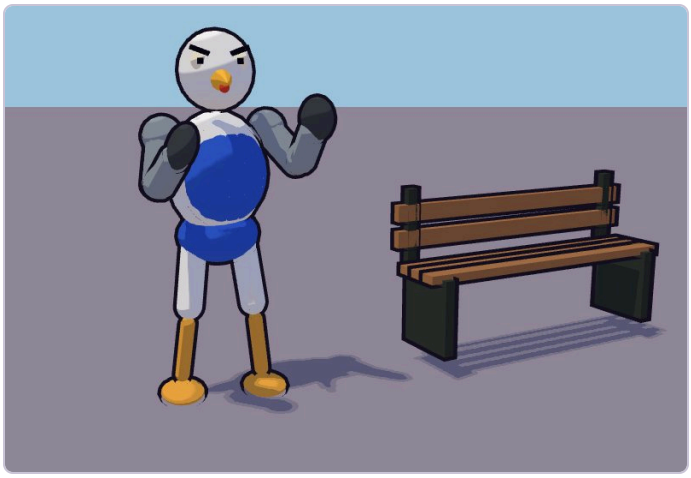
Harold and Mavis, day preset: chunky primitives, warty skin map, floral and curls maps, three hard cel bands, ink hulls.



The same at night: hemisphere and sun drop, a warm lamp and a cool camera-side fill keep the characters readable.



The current thugs (to be replaced by Task C gangs): readable but plain. Your work must reach Harold-level detail.



The kit examples: a gull thug (character interface) and a bench (prop interface, with an atlas decal). The readout shows the budget check.

Palette

#14101f INK: outlines, black	**#6f7f34** Harold skin	**#b8b47a** Harold throat	**#7a5232** Harold cardigan	**#5e3e24** cardigan trim	**#5a5248** Harold slacks	**#7f8f3e** Mavis skin	**#5f73b0** Mavis dress base
#f08ab0 floral pink	**#c8c6c2** Mavis afro	**#6a1f3a** handbag	**#c8a040** gold: clasp, bangle	**#e8d888** toad eye gold	**#ff3d6e** accent pink-red	**#7b4dff** accent violet	**#ff9f1c** accent orange
#23c26b accent green	**#2f5bd3** accent blue	**#ffd23c** accent yellow	**#1fd3c6** accent teal	**#8a4a3a** red brick	**#6f6a62** grey stone	**#9a6a44** tan brick	**#5a4f5c** plum

#4f5a52 green-grey	#a4907a sandstone	#c9b48a trim cream	#eae6dc marble stoop	#ffa050 lamp light	#ffe08a lit window	#ff3a1c neon red	#c8d0ff night camera fill

Use mid-value base colours, because the toon ramp multiplies them by 0.46, 0.78 or 1.06. **Never use pure #000 or #fff**: use INK `#14101f` for black and at most `#f2f2f2` for white. Shadows are tinted slightly cool.

Toon ramp (cel bands)

- The lighting ratio *r* (lit luminance ÷ base luminance) is snapped to three bands: **shadow 0.46** (r below 0.38–0.44), **mid 0.78** and **lit 1.06** (r above 0.86–0.92). Edges are hard (smoothstep over 0.06).
- The shadow band is tinted by (0.88, 0.86, 1.10) at 50%. Strong local light (r above 1.35, such as a lamp at night) passes through unbanded.
- A hit-flash uniform mixes the whole rig to white (`mat.userData.flash`). Every sub-material of a rig must share it.

Ink outlines

- The outlines are **inverted hulls**: back faces of the same geometry pushed out along a mitred vertex direction (`odir`, from `addOdir()`), coloured INK `#14101f`.
- Width = `uW × camera distance`, with the distance clamped to 2.5–36 m. **Characters 0.0042**, **city 0.0034**, **props 0.0030**. That gives a near-constant 2–3 px line on a phone, which fades into the fog colour with distance.
- Only major masses get hulls. Use `outline: false` on parts thinner than about 4 cm (characters) or 6 cm (buildings), and on flat decals. Never use screen-space edge detection.

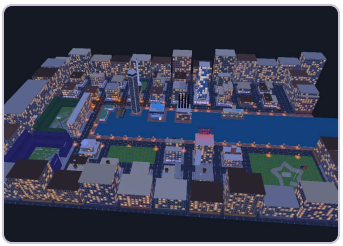
Night lighting rules

Preset	Sky / fog	Hemisphere (sky / ground, intensity)	Sun (colour, intensity, direction)
Day	#9cc4dc / #b4c0c6	#dfe6ff / #8a7f74, 1.45	#fff3dc, 2.1, (−0.5, 1.7, 0.42)
Dusk	#b0706a / #8a6060	#ffb08a / #5a4040, 0.85	#ff9a58, 1.25, (−1, 0.32, −0.25)
Night	#0a0f1a / #0b0f17	#4a5a88 / #14141c, 0.32	moonlight #8aa0e8, 0.45, (0.45, 1.2, −0.9)

- **NIGHT** (0 for day to 1 for night) drives every glow. Windows light by a stable seeded hash (about 45% lit, warm `#ffe08a`/`#ffbf3c`). Shop glass, signs and neon always glow.
- Street lamps are warm orange `#ffa050`: additive glow pools on the ground, billboard halos, and 4 pooled point lights (intensity 16, range 15 m) that follow the hero. Assets **never add their own lights**; they register lamp points instead.
- A cool camera-side fill (`#c8d0ff`, 3.2) keeps characters readable at night. Do not make character bodies emissive; only intentional glow parts (a ghost, crystals, sparks) may glow.



Night street (in-game): lit windows, lamp pools and shop signs (current placeholders; real Baltimore signs are now wanted).


City at night: about 45% of windows lit, warm promenade lamps.



Neon sign quality bar, day: canvas-drawn lettering on a steel lattice.



The same sign at night: red-orange neon, soft glow, water reflection streak.

3 · Code starter and interface

Every delivery is a **drop-in JS module**: no imports, one `META`, build functions that take `K`. Below is a real excerpt from the game, the opening of the thug builder exactly as it ships, followed by the kit examples that define the interface you must follow.

Real excerpt: how the game builds a thug (src/04_actors.js)

```
function buildThug(look) {
  const mat = toonMat(); const d = THUG_D, J = skeleton(d, mat);
  const { jacket, pants, skin } = look;
  part(J.hips, G.sph(0.22, 0.15, 0.17), pants, 0, 0.02, 0);
  part(J.hips, G.cyl(0.235, 0.235, 0.07, 14), '#14101f', 0, 0.1, 0, { outline: false });
  part(J.torso, G.sph(look.big ? 0.33 : 0.27, 0.33, look.big ? 0.24 : 0.2), jacket, 0, 0.3, 0);
  if (!look.big) { part(J.torso, G.box(0.09, 0.5, 0.03), '#14101f', 0.0, 0.3, 0.195, { outline: false }); part
(J.torso, G.cyl(0.16, 0.2, 0.12, 12), jacket, 0, 0.6, 0); }
  else { part(J.torso, G.cyl(0.03, 0.03, 0.5, 6), '#ffd23c', 0, 0.48, 0.2, { rot: [Math.PI / 2 - 0.3, 0, Math.P
I / 2], outline: false }); }
  const head = J.head = grp(J.neck, 0, 0, 0); head.userData.mat = mat;
  part(head, G.cyl(0.08, 0.09, 0.12, 10), skin, 0, 0.03, 0);
  part(head, G.sph(0.17, 0.2, 0.18), skin, 0, 0.2, 0.01);
```

The pattern: one `toonMat` per rig → `skeleton(d, mat)` → `part(joint, G.prim(…), colour, x, y, z, opts)` on each joint → `J.head` group under `J.neck` → `J.kind` → return `J`. The game wraps `J` in a Fighter (position, yaw, HP, AI), drives the joints with the pose tables and flashes `J.mat` on hits.

K member	What it is (reuse it; do not re-create it)
`K.THREE`	three.js r160 namespace (the game's own instance)
`K.toonMat(opts)`	The cel-shaded Lambert material with vertex colours and hit-flash. One per character rig; sub-materials with a map copy its `userData.flash` and `flashC`.
`K.basicMat(opts)`	An unlit material for glow parts (opacity driven by `K.NIGHT.value`).
`K.OUT_CHAR / OUT_PROP / OUT_CITY`	The three shared outline hull materials (widths 0.0042 / 0.0030 / 0.0034).
`K.G.sph / cap / box / cyl / tor`	The primitive geometry makers. They pick HD segment counts for you.
`K.part(parent, geom, colour, x, y, z, {rot, outline, mat, shadow})`	Bakes the colour, adds the ink hull and casts shadows. The parent group must carry `userData.mat`.
`K.grp(parent, x, y, z)` · `K.skeleton(d, mat)`	Groups; the 18-joint rig: tilt, hips, torso, neck, shL/R, elL/R, haL/R, lgL/R, knL/R, ftL/R, swL/R.
`K.HERO_D / K.THUG_D`	Rig proportions (hip height, limb lengths).
`K.blank()` · `legsStance()` · `runLegs()` · `applyPose()`	Pose objects and helpers. `K.THUG_POSES` and `K.HERO_POSES` are the game's own animations.
`K.place(geom, x, y, z, ry)` · `K.makeOut()` · `K.makeAtlas(tiles)`	The static-geometry path: world placement, the merging collector (`out.solid / glow / decal / coll / bld / lamp / hook / build`) and the single canvas atlas.
`K.CITY_MAT` · `K.GLOW_MAT` · `K.LIGHT` · `K.applyLight` · `K.NIGHT` · `K.INK`	The shared city materials, the day/dusk/night presets and the ink colour.

Character interface: example_gull_thug.js (complete)

```
// example_gull_thug.js — the SHAPE every delivered character module must follow (a tiny, deliberately plain ex
ample).
// No imports. Everything comes in through K (toadcity_kit.js). Forward is +Z, up is +Y, 1 unit = 1 metre, feet
on y = 0.
export const META = { id: 'gull_lookout', gang: 'example', kind: 'thug', hp: 8, scale: 1.0, radius: 0.42, weapo
n: null };
```

```
export function buildGullLookout(K, opts = {}) {
  const { G, part, grp, skeleton, toonMat, THUG_D } = K;
  const mat = toonMat();                        // ONE toon material per rig (the game flashes it white on hits)
  const d = THUG_D, J = skeleton(d, mat);       // same joint names + lengths => the game's thug poses drive it
unchanged
  const C = Object.assign({ body: '#e9ecef', wing: '#8d99a6', vest: '#2f5bd3', beak: '#f2b33d', leg: '#e0a040'
}, opts.colors);
  // hips + torso
  part(J.hips, G.sph(0.22, 0.15, 0.17), C.vest, 0, 0.02, 0);
  part(J.torso, G.sph(0.27, 0.33, 0.22), C.body, 0, 0.3, 0);
  part(J.torso, G.sph(0.24, 0.26, 0.12), C.vest, 0, 0.3, 0.1);               // the gang vest (front panel)
  // head: round gull head, hooked beak with the red dot, mean brow
  const head = J.head = grp(J.neck, 0, 0, 0); head.userData.mat = mat;
  part(head, G.sph(0.19, 0.2, 0.2), C.body, 0, 0.2, 0.02);
  part(head, G.cyl(0.0, 0.07, 0.24, 6), C.beak, 0, 0.17, 0.27, { rot: [Math.PI / 2, 0, 0] });
  part(head, G.sph(0.025, 0.025, 0.025), '#d43a2a', 0, 0.14, 0.3, { outline: false });
  for (const s of [-1, 1]) {
    part(head, G.sph(0.045, 0.05, 0.03), '#f6f2d8', s * 0.1, 0.26, 0.16, { outline: false });
    part(head, G.box(0.03, 0.03, 0.02), K.INK, s * 0.1, 0.26, 0.185, { outline: false });
    part(head, G.box(0.1, 0.025, 0.03), K.INK, s * 0.1, 0.31, 0.17, { rot: [0, 0, s * 0.35], outline: false });
  }
  // arms are wings with feather-tip "hands"; legs are thin with webbed feet
  for (const k of ['L', 'R']) {
    part(J['sh' + k], G.sph(0.1, 0.1, 0.1), C.wing, 0, 0, 0);
    part(J['sh' + k], G.cap(0.08, d.upper - 0.08), C.wing, 0, -d.upper / 2, 0);
    part(J['el' + k], G.cap(0.07, d.fore - 0.08), C.wing, 0, -d.fore / 2, 0);
    part(J['ha' + k], G.sph(0.06, 0.12, 0.1), '#3a3f46', 0, -0.05, 0);
    part(J['lg' + k], G.cap(0.07, d.thigh - 0.1), C.body, 0, -d.thigh / 2, 0);
    part(J['kn' + k], G.cap(0.04, d.shin - 0.1), C.leg, 0, -d.shin / 2, 0);
    part(J['ft' + k], G.sph(0.12, 0.04, 0.17), C.leg, 0, -0.03, 0.06);
  }
  J.kind = 'thug'; J.meta = META;
  return J;                                     // return the rig: J.root (THREE.Group), J.mat, the 18 joints,
J.head, J.d
}
```

Rules that make code drop in

- No `import`. No globals, no `window`, no DOM except canvas drawing inside texture or atlas functions. No `Math.random()` at build time (use a seeded LCG from `opts.seed`).
- Characters return the rig `J` (bosses also export `POSES` and `ATTACKS`). Static pieces write only through `out`. Vehicles return `{ root, kind, seats, collider, waterline, animate }`.
- Facing is +Z, the base sits at y = 0 and units are metres. Materials come only from `K`. Never change `toadcity_kit.js`.

4 · Task C: Thug gangs (5 district gangs)

The box below is the **complete copy-paste prompt** for this task. Give it to the model together with `toadcity_starter_kit.zip`. The same text is in `prompts/TASK_C_thug_gangs.md`.

COPY-PASTE PROMPT · TASK C

You are the lead character builder for Toad City Baltimore (Claude Opus, extended thinking on, high or max effort). Build ONE gang per session and self-test it in viewer.html before you move on.

Context (read first)

You are making game assets for **Toad City Baltimore**, a browser open-world brawler set on Baltimore's Inner Harbor. The heroes are Harold, an elderly toad with a cane, and his wife Mavis, who has a grey afro and fights with her handbag. They clean the crime out of each district. Everything is built in code and cel-shaded with ink outlines. The game uses **three.js r160** with plain ES modules: no bundler, no image files, no model files. Every texture is drawn on a canvas at load time.

You were given the **starter kit**: `toadcity_kit.js` (the game's own materials, outline method, rig and pose code, plus the prop collector), `viewer.html` (a standalone test bench), `example_gull_thug.js` (character interface) and `example_bench.js` (prop interface). If the files did not come with this brief, rebuild them from the appendix of the PDF. Your modules must run against the kit unchanged, because the game passes the same `K` object to them.

Conventions (all units are metres):

- Y is up, X is east and Z is south (north is −Z). Facing for a character or vehicle at yaw 0 is **+Z**, and its feet or base sit at y = 0.
- Pavement top (kerb) `CURB = 0.16`. Ground floor height `GF = 4.4`, upper floors `FH = 3.4`, parapet `PAR = 1.3`. Harbour water surface `WY = −0.85`.
- City block: 24 × 24 m. Street: 10 m wide (grid period 34 m). Brick promenade on every water edge: 6 m. Render chunks are 64 m and the collision grid is 16 m.
- Character scale: Harold is about 1.6 m tall, Mavis about 2.0 m to the top of her afro, and a standard thug about 2.0 m. The bruiser is the thug rig scaled ×1.2.

Hard rules for every delivery:

1. **No `import` statements in your modules.** Everything comes in through the `K` argument. Each file has one `export const META` and one or more `export function buildXxx(K, …)`.
2. **Materials:** create them only through `K.toonMat()`, `K.basicMat()` or `K.makeAtlas()`. Outlines must use the shared `K.OUT_CHAR`, `K.OUT_PROP` or `K.OUT_CITY` hulls (which `K.part()` and `out.build()` add for you). Do not use PBR materials, post-processing, edge-detection passes or lights inside assets. Lamps go through `out.lamp()`.
3. **Textures:** draw them on canvas only. Use at most one 1024² atlas per task family and at most 256² per character map. Do not use image files, web fonts or network fetches. Text uses font stacks such as `Impact, "Arial Black", Georgia, serif`.
4. **Deterministic:** do not call `Math.random()` at build time. Use a seeded LCG (`seed = seed * 16807 % 2147483647`) taken from `opts.seed`.
5. **Do not edit `toadcity_kit.js`.** If you need something it lacks, add the helper inside your own module and say so in the README.
6. **Real signage, original characters:** storefronts, signs and props in the Inner Harbor area **may use real Baltimore businesses' names and sign styles where they really exist there**. Examples: Phillips Seafood, Barnes & Noble and Hard Rock Cafe at the Power Plant, the harbour water-taxi branding, Natty Boh-style corner-bar signs, Utz, Old Bay. Recreate the lettering, colours and sign shapes **drawn on canvas or built from geometry**; never copy or embed logo image files, photos or traced artwork. Only put a real name where that business really is or is typical (a crab house, a corner bar, the Power Plant shops); fill everything else with plausible Baltimore signage. **Characters stay original:** no real people, likenesses, real gang colours or symbols, official mascots or film/game characters, and no brand logos on thugs or bosses. Third-party code is allowed only under MIT or CC0, credited with its URL in `LICENSES.md`.
7. Do **not** model landmark buildings (stadiums, the aquarium, the power plant, towers, the fort, the monument and so on). The in-house team builds those.
8. Use three.js r160 APIs only. The code must have no console errors in Chrome or iOS Safari.

Task C: Thug gangs (5 district gangs)

Goal

Build five original street gangs, one per district of the Inner Harbor. Each gang is made of anthropomorphic **Chesapeake animals** and modelled at **Harold-level detail** (the reference is `K.buildHero()`), not at the level of the current simple human thugs. Each gang must drop straight into the game's existing thug system: same skeleton, same poses, same hit-flash and the same outline LOD.

The five gangs (names and animals are approved; looks are yours to design)

id	Gang	Animal	District (landmarks)	Signature accent
`harbor_rats`	Harbor Rats	Norway rats	North shore: Harborplace, USS Constellation, World Trade Center, Washington Monument	mustard `#ffd23c` + navy
`pier_gulls`	Pier Gulls	herring gulls	Piers: Aquarium, Power Plant, Four Seasons	white + `#2f5bd3` blue
`cobble_catfish`	Cobblestone Catfish	channel catfish	Fells Point, Pendry	slate `#5f6688` + `#1fd3c6` teal
`masked_bandits`	Masked Bandits	raccoons	West side: Oriole Park, M&T Bank Stadium, Science Center	`#ff9f1c` orange + grey
`key_hwy_otters`	Key Highway Otters	river otters	South shore: Federal Hill, Domino Sugar, Tide Point, Fort McHenry	`#23c26b` green + brown

Each gang has **4 members**: `grunt_a`, `grunt_b`, `armed` (carries an original improvised weapon: an oar, a crab mallet, a rolled newspaper, a fishing rod, a traffic cone and so on, held in `J.swR`) and `bruiser` (built big; the game scales it ×1.2 and gives it double HP). The members of a gang share a clear gang uniform motif (a jacket colour, bandanas, a patch shape) so the player can tell the districts apart at a glance. Do not use real gang colours or symbols.

Interface (must match exactly)

File: `src/gang_<id>.js`

```
export const GANG = { id: 'harbor_rats', name: 'Harbor Rats', animal: 'rat', district: ['harborplace', 'constellat
ion', 'wtc', 'monument'],
  accent: '#ffd23c', members: ['grunt_a', 'grunt_b', 'armed', 'bruiser'] };
export const META = { grunt_a: { hp: 8, scale: 1, radius: 0.42, weapon: null }, /* … */ bruiser: { hp: 16, scale:
1.2, radius: 0.55, weapon: null, big: true } };
export function buildHarborRat(K, member = 'grunt_a', opts = {}) {   // opts: { lod: 0 | 1, seed, colors }
  const mat = K.toonMat(); const J = K.skeleton(K.THUG_D /* or your own d with the SAME keys */, mat);
  // … K.part(J.<joint>, K.G.<prim>(…), colour, x, y, z, { rot, outline, mat }) …
  J.head = /* group under J.neck */; J.kind = 'thug'; J.meta = META[member];
  return J;
}
```

- Use **the 18 joint names** from `K.skeleton()`: tilt, hips, torso, neck, shL/R, elL/R, haL/R, lgL/R, knL/R, ftL/R and swL/R. The game animates them with `K.THUG_POSES` (idle, run, windup, attack, hurt, down, getup) **without changes**, so the limbs must look right in every one of those poses.
- If you change proportions, pass your own `d` object with the same keys as `K.THUG_D`. Keep `hip` between 0.8 and 1.0.
- `armed` must create `J.swR = K.grp(J.haR, 0, -0.04, 0)` and build the weapon inside it along +Y, as `buildThug` does. The attack pose checks `J.swR`.
- Tails, ears and whiskers can go on extra groups (for example `J.tail`) with an optional `export function animateGang(J, t, state)`, which receives the state ids idle/run/attack/hurt/down. They must still look fine if it is never called.
- Every hull must be the shared `K.OUT_CHAR`; `K.part()` adds it. Use `outline: false` on parts thinner than about 4 cm (whiskers, eyelids, stripes).
- **Far LOD:** `opts.lod === 1` builds the same character with at most 14 parts and no fine details. The game swaps it in beyond 24 m.

Performance budget (per member, HD, `opts.lod = 0`)

- **≤ 40 parts** (meshes, not counting hulls), **≤ 12,000 triangles**, ≤ 3 materials (one `toonMat` plus at most one canvas-mapped `toonMat` sub-material, plus the shared outline).
- LOD 1: ≤ 14 parts, ≤ 3,000 triangles.
- Up to 31 thugs exist at once, so keep the budget.

Style

- Match Harold's construction: chunky primitive shapes (spheres, capsules, cylinders, boxes) built with `K.G`, vertex colours from the palette, cel bands from the kit's toon ramp and ink hulls.
- Big readable heads, a clear facial expression and a gang-specific silhouette (a rat's long snout and tail, a gull's beak, a catfish's barbels, a raccoon's mask, an otter's sleek body).
- Use mid-value base colours. Avoid pure black and white: use `K.INK` (`#14101f`) for black and at most `#f2f2f2` for white. Characters must stay readable under the night preset.
- Skin texture is optional: one canvas map drawn in code (fur flecks, scales), ≤ 256², in the style of the kit's `WARTS` map.

Acceptance checklist

- ☐ `src/gang_<id>.js` for all 5 gangs, each exporting `GANG`, `META` and one build function that takes `member` and `opts`.
- ☐ Every member renders in `viewer.html` in all 7 thug poses with no detached or inverted limbs, and the weapon follows the right hand.
- ☐ Each member measures ≤ 40 parts and ≤ 12k triangles; LOD 1 measures ≤ 14 parts and ≤ 3k triangles; there are ≤ 3 materials.
- ☐ All outlines use the shared `K.OUT_CHAR`; thin parts use `outline:false`; the hulls are BackSide ShaderMaterial children, so the game's distance LOD finds them.
- ☐ The rig faces +Z, the feet touch y = 0 in idle, and the height is 1.8–2.2 m (×1.2 for the bruiser).
- ☐ Hit-flash works: setting `J.mat.userData.flash.value = 0.85` turns the whole rig white, including any sub-materials, which must share `mat.userData.flash` and `flashC`.
- ☐ The four members of a gang read as one gang, and the five gangs read as different from each other at 8 m on a 390×844 screen.
- ☐ Day and night previews for every member, plus a lineup of each gang.
- ☐ Original designs, with no real people, no brand logos on clothing and no real-gang references. `LICENSES.md` is present.

Do NOT

- Do not use skinned meshes, SkinnedMesh/Bone, GLTF, morph targets or mixers. The rig is plain groups posed by code.
- Do not create materials per part. Do not add lights or shadows-only meshes.
- Do not make humans. Every gang member is an animal.
- Do not model the heroes, bosses or landmarks.

Session plan

Build **one gang per session**: one grunt first (self-test it), then the other members, then LOD 1, then the previews. Finish and pass the checklist before you start the next gang.

Delivery and self-test

Make one zip per task: `toadcity_task<LETTER>_<short-name>_v<N>.zip`, laid out like this:

```
src/                  your modules (snake_case file names, e.g. gang_harbor_rats.js, boss_raven_king.js, rowkit_ro
whouse.js)
toadcity_kit.js       the kit, byte-identical to the one you were given
viewer.html           your test bench (start from the kit's viewer.html): imports ./src/*.js, ?show=<id>, ?night=
1, ?pose=…
previews/             <asset_id>_day.png and <asset_id>_night.png at 1280x720, plus <group>_phone_390x844.png
README.md             exports, how to view, the MEASURED budget table (parts, tris, draw calls, materials), known
```

```
issues
LICENSES.md          "All original, procedural" or credits (name, URL, licence) for any MIT / CC0 source used
```

Self-test loop (run it for every asset before you move to the next):

1. Serve the folder (`python3 -m http.server 8000`) and open `viewer.html?show=<id>`. Check day, then press N for night, then P to cycle poses.
2. Read the corner readout. **Draw calls and triangles must be inside the budget** for the asset type. Count your materials too.
3. Look at the asset at gameplay distance in a 390×844 window, the iPhone portrait view. The silhouette must read at about 6–10 m away.
4. Take screenshots. Headless Chrome works with `--use-gl=swiftshader`; wait for `window.__ready === true`, then save the `previews/` PNGs.
5. The console must show zero errors. Fix the asset before you move on.

Do not send: model or image files (GLB, OBJ, FBX, PNG/JPG textures), minified code, copies of three.js, or anything with an unclear licence.

5 · Task D: The 17 bosses

The box below is the **complete copy-paste prompt** for this task. Give it to the model together with `toadcity_starter_kit.zip`. The same text is in `prompts/TASK_D_bosses.md`.

COPY-PASTE PROMPT · TASK D

You are the lead character builder for Toad City Baltimore (Claude Opus, extended thinking on, high or max effort). Build ONE boss per session and self-test it in viewer.html before you move on.

Context (read first)

You are making game assets for **Toad City Baltimore**, a browser open-world brawler set on Baltimore's Inner Harbor. The heroes are Harold, an elderly toad with a cane, and his wife Mavis, who has a grey afro and fights with her handbag. They clean the crime out of each district. Everything is built in code and cel-shaded with ink outlines. The game uses **three.js r160** with plain ES modules: no bundler, no image files, no model files. Every texture is drawn on a canvas at load time.

You were given the **starter kit**: `toadcity_kit.js` (the game's own materials, outline method, rig and pose code, plus the prop collector), `viewer.html` (a standalone test bench), `example_gull_thug.js` (character interface) and `example_bench.js` (prop interface). If the files did not come with this brief, rebuild them from the appendix of the PDF. Your modules must run against the kit unchanged, because the game passes the same `K` object to them.

Conventions (all units are metres):

- Y is up, X is east and Z is south (north is −Z). Facing for a character or vehicle at yaw 0 is **+Z**, and its feet or base sit at y = 0.
- Pavement top (kerb) `CURB = 0.16`. Ground floor height `GF = 4.4`, upper floors `FH = 3.4`, parapet `PAR = 1.3`. Harbour water surface `WY = −0.85`.
- City block: 24 × 24 m. Street: 10 m wide (grid period 34 m). Brick promenade on every water edge: 6 m. Render chunks are 64 m and the collision grid is 16 m.
- Character scale: Harold is about 1.6 m tall, Mavis about 2.0 m to the top of her afro, and a standard thug about 2.0 m. The bruiser is the thug rig scaled ×1.2.

Hard rules for every delivery:

1. **No `import` statements in your modules.** Everything comes in through the `K` argument. Each file has one `export const META` and one or more `export function buildXxx(K, …)`.
2. **Materials:** create them only through `K.toonMat()`, `K.basicMat()` or `K.makeAtlas()`. Outlines must use the shared `K.OUT_CHAR`, `K.OUT_PROP` or `K.OUT_CITY` hulls (which `K.part()` and `out.build()` add for you). Do not use PBR materials, post-processing, edge-detection passes or lights inside assets. Lamps go through `out.lamp()`.
3. **Textures:** draw them on canvas only. Use at most one 1024² atlas per task family and at most 256² per character map. Do not use image files, web fonts or network fetches. Text uses font stacks such as `Impact, "Arial Black", Georgia, serif`.
4. **Deterministic:** do not call `Math.random()` at build time. Use a seeded LCG (`seed = seed * 16807 % 2147483647`) taken from `opts.seed`.
5. **Do not edit `toadcity_kit.js`.** If you need something it lacks, add the helper inside your own module and say so in the README.
6. **Real signage, original characters:** storefronts, signs and props in the Inner Harbor area **may use real Baltimore businesses' names and sign styles where they really exist there**. Examples: Phillips Seafood, Barnes & Noble and Hard Rock Cafe at the Power Plant, the harbour water-taxi branding, Natty Boh-style corner-bar signs, Utz, Old Bay. Recreate the lettering, colours and sign shapes **drawn on canvas or built from geometry**; never copy or embed logo image files, photos or traced artwork. Only put a real name where that business really is or is typical (a crab house, a corner bar, the Power Plant shops); fill everything else with plausible Baltimore signage. **Characters stay original:** no real people, likenesses, real gang colours or symbols, official mascots or film/game characters, and no brand logos on thugs or bosses. Third-party code is allowed only under MIT or CC0, credited with its URL in `LICENSES.md`.
7. Do **not** model landmark buildings (stadiums, the aquarium, the power plant, towers, the fort, the monument and so on). The in-house team builds those.
8. Use three.js r160 APIs only. The code must have no console errors in Chrome or iOS Safari.

Task D: The 17 bosses

Goal

Build the 17 approved bosses: one per landmark district plus the final boss. Each one needs a **model sheet** (front, side, back and three-quarter views, plus the three attack poses on one image) and a **drop-in rig module** in the Toad City style, with **three attack poses** each. Every boss is original: no real people, official mascots, team marks or brand logos on the characters (real signage belongs to the environment, not the bosses).

The roster (approved; keep the names and concepts)

#	id	Boss	Landmark district	Concept
1	`bird_brain_bobby`	Bird Brain Bobby	Oriole Park	A rogue bird mascot (orange-and-black songbird, original design) who pitches fastballs down a brick warehouse hallway
2	`raven_king`	The Raven King	M&T Bank Stadium	A giant raven linebacker in shoulder pads (original colours) with a "Nevermore" blitz charge
3	`dr_shellacious`	Dr. Shellacious	Science Center	A mad snapping-turtle scientist in a lab coat and goggles who fights in the IMAX theatre
4	`hon_queen`	The Hon Queen	Federal Hill	Beehive hair, cat-eye glasses and leopard-print capris; throws hairspray clouds and shoves you down the hill
5	`sugar_rush`	Sugar Rush	Domino Sugar	A sugar-crystal golem, faceted and glittering, glowing at night beside the neon sign
6	`compression_king`	The Compression King	Tide Point (sports store)	An athleisure mannequin brute in blank, unbranded gear
7	`admiral_redclaw`	Admiral Redclaw	Fort McHenry	A redcoat crab admiral from 1812 with a cannon arm firing rocket barrages
8	`big_jimmy`	Big Jimmy	National Aquarium	A giant blue crab (a male "jimmy") fought underwater
9	`voltage_vic`	Voltage Vic	Power Plant	An electric-eel DJ whose attacks drop on a Baltimore club beat
10	`chef_crabcake`	Chef Crabcake	Four Seasons	A chef who throws blinding seasoning and swings two crab mallets
11	`arabber_al`	Arabber Al	Fells Point	A street-produce arabber on a souped-up horse-cart that runs a car chase
12	`pool_shark`	The Pool Shark	Pendry (pier pool deck)	A shark hustler in a cabana shirt and sunglasses, with a pool-cue attack and a belly-flop splash
13	`dragon_boat_duchess`	Dragon Boat Duchess	Harborplace	Rides a giant dragon paddle boat; regal and tiara'd, with a paddle-sceptre
14	`captain_cobblestone`	Captain Cobblestone	USS Constellation	A ghost privateer: translucent glow at night, cutlass and lantern
15	`penthouse_pigeon`	The Penthouse Pigeon	World Trade Center (roof deck)	A rich pigeon in a smoking jacket on a rooftop deck
16	`stoop_scrubber`	The Stoop Scrubber	Washington Monument (Mount Vernon)	A marble-step matriarch with a scrub brush, a bucket and slippery suds
17	`mister_formstone`	**Mister Formstone** (final boss)	City-wide	A crime lord whose face and body are fake-stone (Formstone) armour plates that crack off in phases to reveal the creature beneath

Pool Shark has no further brief, so the line above is the approved direction. Keep the rest of each design consistent with its concept line.

Interface (must match exactly)

File: `src/boss_<id>.js`

```
export const META = { id: 'raven_king', name: 'The Raven King', district: 'mt_stadium', hp: 120, scale: 1.6, radius: 0.9, height: 3.4 };
export const ATTACKS = [ // the game's combat reads these; times are fractions of the attack's duration
  { id: 'nevermore_blitz', pose: 'atk1', dur: 1.1, windup: 0.4, active: [0.4, 0.7], reach: 4.5, arc: 60, dmg: 14, move: 6, fx: 'dash' },
  { id: 'wing_clap',       pose: 'atk2', dur: 0.8, windup: 0.3, active: [0.3, 0.5], reach: 2.2, arc: 160, dmg: 10, fx: 'shockwave' },
  { id: 'talon_slam',      pose: 'atk3', dur: 1.3, windup: 0.6, active: [0.6, 0.75], reach: 3.0, arc: 360, dmg: 16, fx: 'ground_slam' },
];
export function buildRavenKing(K, opts = {}) {
  const mat = K.toonMat(); const J = K.skeleton(D /* same 18 joint keys */, mat);
  // … K.part(…) …   optional extra groups: J.extra = { wingL, wingR, tail, … }
  J.kind = 'boss'; J.meta = META; return J;
}
export const POSES = {             // each returns a K.blank()-format object: { y, ty, tilt:[x,y,z], hips:[…], …, swR:[…] } (radians)
  idle(t, J) { /* … */ }, run(t, speed, J) { /* … */ }, hurt(u, J) {}, down(u, ground, J) {},
  atk1(u, J) { /* u = 0..1 through the attack */ }, atk2(u, J) {}, atk3(u, J) {},
};
export function animateBoss(J, t, state, u) { /* optional: extra groups (wings, tails, suds, glow); must be safe to skip */ }
```

- Poses are applied with `K.applyPose(J, pose, rate, dt, snapTilt)`, exactly like the heroes. Build each pose from `K.blank()` and helpers such as `K.legsStance()` and `K.runLegs()`, and look at `K.HERO_POSES.atk1–atk3` for the style: anticipation, a fast strike and a held follow-through.
- **Three attack poses per boss** must read as three distinct silhouettes (for example a charge, a sweep and a slam) and match the concept. The hit window `active` must line up with the visible strike frame.
- Vehicle-bound bosses (Arabber Al's cart, the Duchess's dragon boat) put the rider and the vehicle in the same module: `J.vehicle` is a Group under `J.root`, built with the same material rules.
- Night: glowing elements (Sugar Rush's crystals, Voltage Vic's sparks, Captain Cobblestone's ghost glow) use `K.basicMat({ transparent: true })` parts whose opacity follows `K.NIGHT.value` in `animateBoss`. They must not add lights.
- Mister Formstone: `opts.phase` 0/1/2 removes armour plates in stages, or `J.plates` lists the plate meshes so the game can knock them off.

Model sheet (per boss)

`previews/<id>_sheet.png` at 2048×1024 with turnaround views (front, ¾, side, back) plus atk1, atk2 and atk3, rendered from the viewer at a neutral grey background in the day preset. Add a name label under each view. Also deliver `<id>_day.png`, `<id>_night.png` and `<id>_phone_390x844.png` (the boss facing Harold at about 6 m).

Performance budget (per boss)

- **≤ 70 parts**, **≤ 22,000 triangles**, ≤ 4 materials (toon plus up to 2 mapped toon sub-materials plus 1 basic glow material), and the shared `K.OUT_CHAR`.
- Only one boss is on screen at a time, but it fights alongside up to 6 thugs and the two heroes on an iPhone.

Acceptance checklist

- ☐ `src/boss_<id>.js` for all 17 bosses, each exporting `META`, `ATTACKS` (exactly 3), `POSES` (idle, run, hurt, down, atk1–3) and one build function. `animateBoss` is optional.
- ☐ Model sheet plus day, night and phone previews for each boss.
- ☐ ≤ 70 parts, ≤ 22k triangles and ≤ 4 materials, measured and written in the README table.
- ☐ Faces +Z, feet on y = 0, `META.height` matches the measured bounding box within 10%.
- ☐ Hit-flash turns the whole boss white (every sub-material shares `mat.userData.flash`).
- ☐ The three attacks are visually distinct and the `active` windows match the strike frames.
- ☐ Every boss is an original design that matches the concept line, with no real people, likenesses, mascots, team marks or brands.
- ☐ No console errors. Poses never invert joints or detach parts.

Do NOT

- Do not use SkinnedMesh, GLTF, morph targets, AnimationMixer or image files.

- Do not build landmark architecture for the boss arenas (the in-house team does that). A small boss-owned prop (a lectern, a DJ booth, a bucket) is fine and counts toward the budget.
- Do not make likenesses of real persons or official mascots.

Session plan

Build **one boss per session**: (1) a written design note of about 120 words on silhouette, palette and the three attacks; (2) the build plus idle, self-tested; (3) the three attacks, self-tested; (4) hurt, down and run; (5) the model sheet and previews; (6) the checklist. Mister Formstone goes last.

Delivery and self-test

Make one zip per task: `toadcity_task<LETTER>_<short-name>_v<N>.zip`, laid out like this:

```
src/                  your modules (snake_case file names, e.g. gang_harbor_rats.js, boss_raven_king.js, rowkit_ro
whouse.js)
toadcity_kit.js       the kit, byte-identical to the one you were given
viewer.html           your test bench (start from the kit's viewer.html): imports ./src/*.js, ?show=<id>, ?night=
1, ?pose=…
previews/             <asset_id>_day.png and <asset_id>_night.png at 1280x720, plus <group>_phone_390x844.png
README.md             exports, how to view, the MEASURED budget table (parts, tris, draw calls, materials), known
issues
LICENSES.md           "All original, procedural" or credits (name, URL, licence) for any MIT / CC0 source used
```

Self-test loop (run it for every asset before you move to the next):

1. Serve the folder (`python3 -m http.server 8000`) and open `viewer.html?show=<id>`. Check day, then press N for night, then P to cycle poses.
2. Read the corner readout. **Draw calls and triangles must be inside the budget** for the asset type. Count your materials too.
3. Look at the asset at gameplay distance in a 390×844 window, the iPhone portrait view. The silhouette must read at about 6–10 m away.
4. Take screenshots. Headless Chrome works with `--use-gl=swiftshader`; wait for `window.__ready === true`, then save the `previews/` PNGs.
5. The console must show zero errors. Fix the asset before you move on.

Do not send: model or image files (GLB, OBJ, FBX, PNG/JPG textures), minified code, copies of three.js, or anything with an unclear licence.

6 · Delivery instructions

The full delivery and self-test block is the last part of every copy-paste prompt above. In summary: **one zip per task**, named `toadcity_task<LETTER>_<short-name>_v<N>.zip`, containing `src/` (your modules), an unchanged `toadcity_kit.js`, `viewer.html`, `previews/` (day and night PNGs at 1280×720 per asset, plus a 390×844 phone framing), a `README.md` with the measured budget table, and `LICENSES.md`.

> **Licences.** Everything should be procedural or original. If any snippet comes from elsewhere, it must be MIT or CC0 and credited in `LICENSES.md` (name, URL, licence). three.js itself is MIT and is loaded by the viewer only, never copied into `src/`. Do not send anything with an unclear licence, and no photographs, logo image files or traced artwork. Real sign wording and styles are fine when redrawn in code.

Hand-back: one zip per task, versioned (`_v1`, `_v2` …); re-deliver the whole zip after fixes, not a diff. A separate QA model reviews each zip (pass/fail per asset, with numbered fixes), so expect one fix round. List anything unfinished under *Known issues* in the README.

Appendix · Kit core (characters)

The character-relevant core of `toadcity_kit.js`, printed so you can rebuild it if the zip is missing (… marks omitted sections: buildHero, buildThug, hero poses, the prop collector; THUG_POSES.down reuses HERO_POSES.down). The file ends by exporting every function and constant above as `export const K = { … }`. Use the zip copy when you have it.

```
// toadcity_kit.js — Toad City Baltimore: the shared character / prop kit (HD cel-shaded path only).
// This is a faithful standalone copy of the game's own materials, outline method and rig code (three.js r160),
// so modules built against it drop straight into the game. Deliveries NEVER import anything: they receive this object as `K`.
import * as THREE from 'three';

const TAU = Math.PI * 2;
const clamp = (v, a, b) => (v < a ? a : v > b ? b : v);
const lerp = (a, b, t) => a + (b - a) * t;
const damp = (a, b, r, dt) => a + (b - a) * (1 - Math.exp(-r * dt));
const wrapA = (a) => { a = (a + Math.PI) % TAU; if (a < 0) a += TAU; return a - Math.PI; };
const ease = (t) => t * t * (3 - 2 * t);
const QUAL = { hd: true, outlines: [], texs: [] };   // the game runs HD by default; the kit is HD-only
const INK = '#14101f';                               // the one ink colour: outlines, shadows on signs, dark trims
const NIGHT = { value: 0 };                          // 0 day .. 1 night (the game drives it; viewers set it)

// ---- toon (cel) material: MeshLambert + vertex colours + 3 hard bands + a hit-flash uniform ----
function toonMat(opts = {}) {
  const m = new THREE.MeshLambertMaterial(Object.assign({ vertexColors: true }, opts));
  m.userData.flash = { value: 0 }; m.userData.flashC = { value: new THREE.Color(1, 1, 1) };
  m.onBeforeCompile = (sh) => {
    sh.uniforms.uFlash = m.userData.flash; sh.uniforms.uFlashC = m.userData.flashC;
    sh.fragmentShader = sh.fragmentShader
      .replace('#include <common>', '#include <common>\nuniform float uFlash; uniform vec3 uFlashC;')
      .replace('#include <opaque_fragment>', `
        { // HD cel bands: shadow 0.46 / mid 0.78 / lit 1.06 of the base colour, hard edges; shadows tinted cool
          float ld = max(dot(diffuseColor.rgb, vec3(0.299, 0.587, 0.114)), 1e-3), lo = dot(outgoingLight, vec3(0.299, 0.587, 0.114)), r = lo / ld;
          float b = mix(0.46, 0.78, smoothstep(0.38, 0.44, r)); b = mix(b, 1.06, smoothstep(0.86, 0.92, r)); b = mix(b, r, step(1.35, r));
          outgoingLight *= b / max(r, 1e-3);
          outgoingLight = mix(outgoingLight, outgoingLight * vec3(0.88, 0.86, 1.1), (1.0 - smoothstep(0.38, 0.44, r)) * 0.5); }
        outgoingLight = mix(outgoingLight, uFlashC, uFlash);
        #include <opaque_fragment>`);
  };
  m.customProgramCacheKey = () => 'psx' + (opts.map ? 'M' : '');
  return m;
}
// unlit material for neon, glows and lamp heads (night-only things fade with opacity)
function basicMat(opts) { return new THREE.MeshBasicMaterial(opts); }
// ---- ink outlines: inverted hulls. Back faces pushed out along a mitred direction (attribute `odir`), width ~constant on screen ----
const OUT_U = { fogC: { value: new THREE.Color('#000') }, fogN: { value: 100 }, fogF: { value: 200 } };
function outlineMat(w) {
  return new THREE.ShaderMaterial({
    uniforms: { uW: { value: w }, uC: { value: new THREE.Color(INK) }, uFogC: OUT_U.fogC, uFogN: OUT_U.fogN, uFogF: OUT_U.fogF },
    vertexShader: `attribute vec3 odir; uniform float uW; varying float vD;
      void main() { vec4 wp = modelMatrix * vec4(position, 1.0); vec3 wd = mat3(modelMatrix) * odir;
        float d = distance(wp.xyz, cameraPosition); wp.xyz += wd * uW * clamp(d, 2.5, 36.0);
        vD = d; gl_Position = projectionMatrix * viewMatrix * wp; }`,
    fragmentShader: `uniform vec3 uC; uniform vec3 uFogC; uniform float uFogN; uniform float uFogF; varying float vD;
      void main() { gl_FragColor = vec4(mix(uC, uFogC, smoothstep(uFogN, uFogF, vD)), 1.0);
        #include <colorspace_fragment>
      }`,
    side: THREE.BackSide,
  });
}
// the three shared outline widths: city 0.0034, characters 0.0042, props 0.0030 (fraction of camera distance, clamped 2.5..36 m)
const OUT_CITY = outlineMat(0.0034), OUT_CHAR = outlineMat(0.0042), OUT_PROP = outlineMat(0.0030);
function addOdir(g) {
  const p = g.attributes.position, n = g.attributes.normal, map = new Map(), keys = [];
  for (let i = 0; i < p.count; i++) {
    const k = Math.round(p.getX(i) * 1e3) + ',' + Math.round(p.getY(i) * 1e3) + ',' + Math.round(p.getZ(i) * 1e3);
    keys.push(k); let e = map.get(k); if (!e) map.set(k, (e = []));
    const nx = n.getX(i), ny = n.getY(i), nz = n.getZ(i);
    if (!e.some((q) => q[0] * nx + q[1] * ny + q[2] * nz > 0.985)) e.push([nx, ny, nz]);
  }
  const res = new Map();
  for (const [k, e] of map) {
    let x = 0, y = 0, z = 0; for (const q of e) { x += q[0]; y += q[1]; z += q[2]; }
    const l = Math.hypot(x, y, z) || 1; x /= l; y /= l; z /= l;
    let mn = 1; for (const q of e) mn = Math.min(mn, q[0] * x + q[1] * y + q[2] * z);
    const s = 1 / Math.max(mn, 0.35); res.set(k, [x * s, y * s, z * s]);
  }
  const o = new Float32Array(p.count * 3);
  for (let i = 0; i < p.count; i++) { const d = res.get(keys[i]); o[i * 3] = d[0]; o[i * 3 + 1] = d[1]; o[i * 3 + 2] = d[2]; }
  g.setAttribute('odir', new THREE.BufferAttribute(o, 3));
  return g;
}
function bakeColor(g, col) {
  const c = new THREE.Color(col), n = g.attributes.position.count, a = new Float32Array(n * 3);
  for (let i = 0; i < n; i++) { a[i * 3] = c.r; a[i * 3 + 1] = c.g; a[i * 3 + 2] = c.b; }
  g.setAttribute('color', new THREE.BufferAttribute(a, 3)); return g;
}
// canvas -> texture (the game draws every texture on a canvas at load: no image files). HD = full-size source, trilinear mipmaps.
function lowTex(src) {
  const t = new THREE.CanvasTexture(src); t.colorSpace = THREE.SRGBColorSpace; t.minFilter = THREE.LinearMipmapLinearFilter; t.magFilter = THREE.Line
arFilter;
  t.generateMipmaps = true; t.anisotropy = 4; t.userData.src = src; QUAL.texs.push(t); return t;
}

// …
```

```
// ---------------- characters: procedural stylised rigs (groups of toon parts with outline hulls) ----------------
const G = { // low-poly on purpose (32-bit era budgets)
  sph: (rx, ry, rz, ws = 8, hs = 6) => new THREE.SphereGeometry(1, QUAL.hd ? Math.max(ws * 2, 18) : Math.min(ws, 9), QUAL.hd ? Math.max(hs * 2, 12) :
Math.min(hs, 7)).scale(rx, ry, rz),
  cap: (r, len, rs = 6) => new THREE.CapsuleGeometry(r, len, QUAL.hd ? 4 : 2, QUAL.hd ? 12 : Math.min(rs, 6)),
  box: (w, h, d) => new THREE.BoxGeometry(w, h, d),
  cyl: (rt, rb, h, s = 7, open = false) => new THREE.CylinderGeometry(rt, rb, h, QUAL.hd ? Math.max(s * 2, 14) : Math.min(s, 8), 1, open),
  tor: (R, r, arc = TAU) => new THREE.TorusGeometry(R, r, QUAL.hd ? 6 : 3, QUAL.hd ? 16 : 7, arc),
};
function part(parent, geom, col, x = 0, y = 0, z = 0, o = {}) {
  bakeColor(geom, col); const ol = o.outline !== false && geom.attributes.normal;
  if (ol) addOdir(geom);
  const m = new THREE.Mesh(geom, o.mat || parent.userData.mat || parent.parent?.userData.mat);
  m.position.set(x, y, z); if (o.rot) m.rotation.set(o.rot[0], o.rot[1], o.rot[2]);
  if (ol) { const h = new THREE.Mesh(geom, o.olMat || OUT_CHAR); h.castShadow = false; h.visible = QUAL.hd; m.add(h); QUAL.outlines.push(h); }
  geom.computeBoundingSphere(); m.castShadow = o.shadow !== false && geom.boundingSphere.radius > 0.1; parent.add(m);
  return m;
}
function grp(parent, x = 0, y = 0, z = 0) { const g = new THREE.Group(); g.position.set(x, y, z); parent.add(g); return g; }

// skeleton: root (world pos + facing) > tilt (falls / flips / spins) > hips > torso > neck; legs on hips, arms on torso
function skeleton(d, mat) {
  const J = {}; J.root = new THREE.Group(); J.mat = mat;
  const P = (g) => { g.userData.mat = mat; return g; };
  J.tilt = P(grp(J.root)); J.hips = P(grp(J.tilt, 0, d.hip, 0)); J.torso = P(grp(J.hips, 0, d.torsoY, 0)); J.neck = P(grp(J.torso, 0, d.neckY, 0));
  for (const [s, k] of [[1, 'L'], [-1, 'R']]) {
    J['lg' + k] = P(grp(J.hips, s * d.legX, 0, 0)); J['kn' + k] = P(grp(J['lg' + k], 0, -d.thigh, 0)); J['ft' + k] = P(grp(J['kn' + k], 0, -d.shin,
0));
    J['sh' + k] = P(grp(J.torso, s * d.shX, d.shY, 0)); J['el' + k] = P(grp(J['sh' + k], 0, -d.upper, 0)); J['ha' + k] = P(grp(J['el' + k], 0, -d.for
e, 0));
  }
  J.d = d; return J;
}
const HERO_D = { hip: 0.64, torsoY: 0.05, neckY: 0.58, legX: 0.17, thigh: 0.3, shin: 0.29, shX: 0.37, shY: 0.42, upper: 0.25, fore: 0.23 };
// …
const THUG_D = { hip: 0.92, torsoY: 0.06, neckY: 0.64, legX: 0.13, thigh: 0.42, shin: 0.41, shX: 0.31, shY: 0.52, upper: 0.3, fore: 0.27 };
// …
const JOINTS = ['tilt', 'hips', 'torso', 'neck', 'shL', 'shR', 'elL', 'elR', 'haL', 'haR', 'lgL', 'lgR', 'knL', 'knR', 'ftL', 'ftR', 'swL', 'swR'];
function blank() { const p = { y: 0, ty: 0 }; for (const j of JOINTS) p[j] = [0, 0, 0]; return p; }
function legsStance(p, wide = 0.13, bend = 0.3) {
  p.lgL = [-bend * 0.5, 0, wide]; p.lgR = [-bend * 0.3, 0, -wide]; p.knL = [bend, 0, 0]; p.knR = [bend * 0.9, 0, 0];
  p.ftL = [-bend * 0.5, 0, -wide]; p.ftR = [-bend * 0.5, 0, wide]; p.y = -0.04 - bend * 0.08;
}
function runLegs(p, ph, amp = 1) {
  for (const [k, o] of [['L', 0], ['R', Math.PI]]) {
    const s = Math.sin(ph + o), c = Math.cos(ph + o);
    p['lg' + k] = [-0.85 * s * amp - 0.15, 0, 0]; p['kn' + k] = [0.25 + 1.25 * Math.max(0, c) * amp, 0, 0]; p['ft' + k] = [0.3 * s * amp, 0, 0];
  }
  p.y = -0.05 + 0.07 * Math.abs(Math.sin(ph)) * amp;
}
// hero sword grip: sword group rotation relative to the hand (blade points along the fist's forward)
const SWF = [Math.PI / 2, 0, 0];
// …
const THUG_POSES = {
  idle(t, J) {
    const p = blank(); legsStance(p, 0.12, 0.25); const b = Math.sin(t * 4) * 0.03; p.y += b;
    p.torso = [0.08, 0.35, 0]; p.neck = [0, -0.35, 0]; p.hips = [0, -0.2, 0];
    p.shL = [-0.9, 0, 0.25]; p.elL = [-1.7, 0, 0]; p.shR = [-0.55, 0, -0.35]; p.elR = [-1.9, 0, 0];
    if (J.swR) { p.shR = [-0.3, 0, -0.2]; p.elR = [-0.8, 0, 0]; p.swR = [Math.PI / 2 - 0.5, 0, 0]; }
    return p;
  },
  run(t, sp, J) {
    const p = blank(), ph = t * (6 + sp); runLegs(p, ph, clamp(sp / 4.5, 0.4, 1));
    p.torso = [0.22, Math.sin(ph) * 0.2, 0];
    p.shL = [0.8 * Math.sin(ph), 0, 0.15]; p.shR = [-0.8 * Math.sin(ph), 0, -0.15]; p.elL = p.elR = [-1.1, 0, 0];
    if (J.swR) p.swR = [Math.PI / 2 - 0.4, 0, 0];
    return p;
  },
  windup(u, J) {
    const p = THUG_POSES.idle(0, J);
    if (J.swR) { p.shR = [-2.9, 0, -0.3]; p.elR = [-0.7, 0, 0]; p.torso = [-0.2, -0.4, 0]; p.swR = [Math.PI / 2 - 0.2, 0, 0]; }
    else { p.torso = [0.05, -0.75, 0]; p.shR = [-0.2, 0, -0.3]; p.elR = [-2.2, 0, 0]; p.hips = [0, -0.4, 0]; }
    p.lgL = [-0.5, 0, 0.12]; p.knL = [0.5, 0, 0]; return p;
  },
  attack(u, J) {
    const p = THUG_POSES.idle(0, J); const a = ease(clamp(u / 0.35, 0, 1));
    if (J.swR) { p.shR = [lerp(-2.9, -0.5, a), 0, -0.1]; p.elR = [lerp(-0.7, 0, a), 0, 0]; p.torso = [lerp(-0.2, 0.5, a), 0.2, 0]; p.swR = [Math.PI /
2 - 0.2, 0, 0]; }
    else { p.torso = [0.15, lerp(-0.75, 0.6, a), 0]; p.shR = [lerp(-0.2, -1.6, a), 0, 0.1]; p.elR = [lerp(-2.2, -0.05, a), 0, 0]; p.hips = [0, 0.3,
0]; }
    p.lgL = [-0.75, 0, 0.12]; p.knL = [0.6, 0, 0]; p.lgR = [0.4, 0, -0.1]; return p;
  },
  hurt(u, J) {
    const p = blank(); legsStance(p, 0.14, 0.45); const k = 1 - u;
    p.torso = [-0.55 * k, 0.3 * k, 0]; p.neck = [-0.6 * k, 0, 0]; p.shL = [-0.6, 0, 0.8]; p.shR = [-0.6, 0, -0.8]; p.elL = p.elR = [-0.9, 0, 0]; retu
rn p;
  },
  down(u, ground) {
    const p = HERO_POSES.down(u, ground); p.ty = ground ? 0.2 : 0.5; return p;
  },
  getup(u, J) { const p = THUG_POSES.idle(0, J); legsStance(p, 0.2, 1.2 * (1 - u)); p.torso = [0.9 * (1 - u), 0, 0]; return p; },
};
// apply a pose to a rig, damped (rate r); tilt angles are applied raw when `snapTilt`
function applyPose(J, p, r, dt, snapTilt) {
  for (const j of JOINTS) {
    const g = J[j]; if (!g) continue; const t = p[j];
    if (j === 'tilt' && snapTilt) { g.rotation.set(t[0], t[1], t[2]); continue; }
    if (j === 'tilt') { g.rotation.x = wrapA(g.rotation.x); g.rotation.y = wrapA(g.rotation.y); }
    g.rotation.x = damp(g.rotation.x, t[0], r, dt); g.rotation.y = damp(g.rotation.y, t[1], r, dt); g.rotation.z = damp(g.rotation.z, t[2], r, dt);
  }
  J.hips.position.y = damp(J.hips.position.y, J.d.hip + p.y, r, dt);
  J.tilt.position.y = damp(J.tilt.position.y, p.ty, r, dt);
}
```